Messaging: the inbox in two statements, and where the next 30 ms are

What was built, what the runs showed and what they uncovered. Branch `feat/messaging`, second session of 11 October 2026, step 7 of the tuning order in `doc/messaging-architecture.md` section 6.

1. Summary

	Before	After
Statements per inbox page	1 + one per conversation on the page: 21 for a page of 20, 51 for a page of 50	2 whatever the size: the page, then the participants of every conversation on it in one IN query
Inbox page of 20, four concurrent readers	1.5 s at the median, 1.9 s at p95	165 ms at the median, 222 ms at p95
Inbox page of 50	3.4 to 4.3 s	182 ms
Pages served per second by one node	1.4	20
Support queue (staff)	same shape, same cost	same change, same gain
What a read costs, measured	not measured	new timer `messaging.query.latency{query}` on every read of the read model
Uncovered		a statement MySQL runs in under a millisecond costs the service about 30 ms alone and 80 ms under four readers; new step 9

The change is confined to `ConversationQueries` (application layer of the messaging service) plus one meter; no schema, index or API change. The branch is not committed.

2. What was changed

An inbox page is read from the MySQL read model: one statement selects the page of conversation rows the caller takes part in, latest activity first, and the view of each row needs its participants (to show who is in the conversation and to compute the caller's unread count from the participant's read marker). Until now the participants were read one conversation at a time, after each other, so a page of 20 cost 21 statements and the page size set the latency. The support queue read by staff had the same shape.

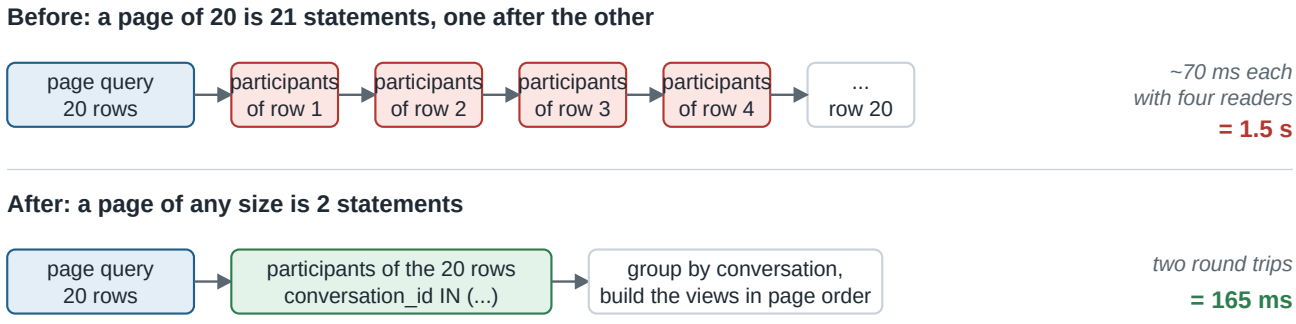

Figure 1. The statements of one inbox page before and after this step.

Now the page is collected first, then one query reads the participants of every conversation on it (`conversation_id IN (...)`, served by the prefix of the unique key on the participant table), the rows are grouped by conversation in memory and the views are built in the order of the page. An empty page asks for nothing more. The view of a single conversation (`GET /api/conversations/{id}`, the socket handshake) keeps its one participants query; the function that turns a row and its participants into the view is shared by both paths.

Every public read of `ConversationQueries` is now timed as `messaging.query.latency` with the tag `query` (`inbox`, `by_status`, `conversation`, `messages`) and `outcome`, from the subscription to the complete answer. It is the cost of a REST read minus the HTTP hop, and the number this step is judged by.

3. Tests

- `ConversationQueriesInboxTest` (new, mocked mappers): a page of three conversations issues exactly one participants query whose IN list is the page's ids; the views come back in page order although the query answers in id order; `unread` is computed for the viewer alone (30, 0 and 15 for three read markers); an empty page issues no participants query; the staff queue is batched the same way and shows no unread for an outsider; a conversation without participant rows still renders; the timer counts one sample per page.
- `ConversationQueriesAttachTest` and the REST slice test keep passing on the single-conversation path.
- `MessagingEndToEndTest` reads the inbox over REST for a member and for staff as before; it covers the batched path against the live read model.
- `InboxLoadDriver` (new, `src/test/.../load`): one member in 50 support conversations and one staff member who joined them all, then N clients fetching the member's inbox, the staff inbox and the staff queue with pages of 20 and 50; reports p50/p95/p99/max per query and page size and the `messaging.query` and pool lines of the scrape. Run with `-Dinbox.*` as documented in its header.

4. Measurements

One node on port 8098, a throwaway EventSourcingDB started empty before every run, the development Kafka and the live MySQL 8.0.34, all on the same 8-core host as the driver. A = the committed code before this step, B = the batched queries; four clients, 60 s, pages of 20 and 50 over 50 conversations. Latency is the driver's, request to parsed answer, p50 / p95 / max.

	A: one query per row	B: one query per page
requests answered in 60 s	83 = 1.4 requests/s	1,207 = 20.1 requests/s
member inbox, page of 20	1,497 / 1,862 / 1,862 ms	165 / 222 / 320 ms
member inbox, page of 50	3,449 / 5,054 / 5,054 ms	182 / 257 / 386 ms
staff inbox, page of 20	1,768 / 2,224 / 2,224 ms	167 / 225 / 299 ms
staff inbox, page of 50	4,311 / 5,053 / 5,053 ms	182 / 254 / 414 ms
staff queue, page of 20	1,755 / 2,124 / 2,124 ms	164 / 231 / 318 ms
staff queue, page of 50	3,518 / 4,529 / 4,529 ms	180 / 244 / 395 ms
`messaging.query.latency{inbox}` p50 / p95 / p99, server side	not measured before this step	180 / 239 / 315 ms

The page size no longer matters (a page of 50 costs a tenth more than a page of 20, not twice) and a page is nine times faster at the median. One request of about 1,200 in each run waited 30 s on the driver's side

while the server had answered it within 0.4 s, its maximum; a client-side artefact, not counted.

4.1 Where the remaining 165 ms are

MySQL executes each of the statements in well under a millisecond: `EXPLAIN` on the loaded rows shows the inbox join reading the participant by `uk_participant`, the IN query by the same key and the queue page by `idx_conv_status_last`, and profiling gives 0.3 to 0.7 ms per statement. Yet two statements cost the service 165 ms with four clients, and the two statements of a single-conversation read cost 75 ms with the node otherwise idle. Three 30 s probes with one client (driver p50, p95 in brackets):

Node	member inbox, page of 20	member inbox, page of 50	requests/s
as configured (r2dbc-mysql with TLS, its default when the server offers it)	58 (269) ms	69 (202) ms	11.3
`sslMode=DISABLED`	59 (78) ms	71 (95) ms	14.7
`tcpNoDelay=true`	95 (128) ms	120 (176) ms	8.8

With one client a page costs about 60 ms, 30 ms per statement. It is not TLS (the same median without it; the tail is tighter, and the setting stays as it is because the development host is not the production shape) and not a Nagle or delayed-ACK stall (no 40 ms signature, and forcing `TCP_NODELAY` does not help; that run was slower across the board, host noise). The cost sits in the service's own path from the mapper call to the mapped row: reactive-mybatis-support's statement execution and result mapping over the pooled connection. It is the same cost that makes the read-model projection spend about 40 ms per event, so finding where those 30 ms go (a scheduler hop per statement, a connection acquire with validation, the result mapping, logging) pays on every read and every projected event. That is step 9 of the tuning order.

5. Operating notes

- **Page size.** `messaging.max-page-size` (200) can stay: a page of 200 is still two statements, and the IN list of 200 ids is well within MySQL's limits. The payload is what grows.
- **Timer.** `messaging.query.latency{query="inbox"}` p50 is the number to watch after a deploy; it is the REST latency minus the HTTP hop. A rejected read (404, 403) is a sample with `outcome=rejected`.
- **Indexes.** No change: the participant read of the inbox join and the IN query both use the unique key on (conversation_id, participant_type, participant_id); the page sort of the inbox (latest activity first) is a filesort over the caller's own conversations, a few dozen rows.
- **Clean-up of load runs.** The multi-table DELETE of the run scripts needs `-D announce_reactive_db` on the mysql client (without it the statement fails silently and the `load %` rows stay); the scripts are fixed and the rows of today's runs are gone.

6. Files changed

Paths under `messaging/src/main/java/org/linlinjava/announce/messaging/` unless noted.

File	Change
application/ConversationQueries.java	page views built from one IN query for the participants; shared view function; every read timed
infrastructure/metrics/MessagingMeters.java	QUERY_LATENCY, TAG_QUERY, queryLatency()
test: application/ConversationQueriesInboxTest.java	new: one query per page, page order, unread per viewer, empty page
test: application/ConversationQueriesAttachTest.java	constructor takes a meter registry
test: load/InboxLoadDriver.java	new: inbox and queue read load against a running node
doc/messaging-architecture.md, doc/messaging-context.md	sections 2.2, 4.4, 5, 6 (step 7 done, new step 9) and the new 7e; context sections 4 and 7

Branch state: uncommitted on `feat/messaging`, on top of this morning's sockets commit. Next in the tuning order: step 8 (command-side throughput) and the new step 9 (the per-statement cost), which share a cause if the command path's bounded-elastic hop is what the statement path pays too.